

NOME DA INSTITUIÇÃO DE ENSINO
CURSO DE [NOME DO CURSO]
DISCIPLINA DE SISTEMAS OPERACIONAIS

Linux Mint em Ambiente Docker

Estrutura do Sistema Operacional, Kernel e Automação via
Shell Script

Avaliação AV2

Equipe:

Nome do Integrante 1
Nome do Integrante 2
Nome do Integrante 3
Nome do Integrante 4
Nome do Integrante 5

Maceió/AL
2026

Sumário

1	Introdução	2
2	Ambiente do Projeto: Docker	2
2.1	O que é o Docker	2
2.2	Configuração do Linux Mint no Docker	3
3	O Sistema Operacional Linux Mint	4
3.1	Visão geral	4
3.2	Estrutura do Sistema Operacional	4
3.2.1	Hierarquia de diretórios (FHS)	4
4	O Kernel Linux	5
4.1	Definição e tipo de kernel	5
4.2	Principais funções do Kernel	5
4.3	Observando o Kernel na prática	6
5	Linguagem de Programação Utilizada: Shell Script	7
5.1	Justificativa	7
5.2	Scripts desenvolvidos	7
6	Versionamento com Git e GitHub	8
7	Conclusão	9
8	Referências	9

1 Introdução

Este relatório foi desenvolvido como parte da Avaliação AV2 da disciplina de Sistemas Operacionais, cujo objetivo é o estudo prático e teórico de um sistema operacional executado em ambiente virtualizado por containers. O sistema operacional escolhido pela equipe foi o **Linux Mint**, uma distribuição Linux baseada no Ubuntu/Debian, amplamente utilizada em desktops por sua estabilidade, leveza e familiaridade de uso.

O trabalho contempla os seguintes pontos, conforme especificado nas regras da atividade:

- Instalação do sistema operacional escolhido em um ambiente containerizado (Docker);
- Apresentação da estrutura geral do sistema operacional;
- Explicação sobre o funcionamento do Kernel;
- Demonstração prática utilizando uma linguagem de script (Shell Script);
- Versionamento de todo o material produzido em um repositório do GitHub;
- Elaboração deste relatório em $\text{\LaTeX}$.

Nas seções a seguir, será apresentada a justificativa para o uso do Docker, uma visão geral do Linux Mint, a estrutura de diretórios do sistema operacional (FHS), o funcionamento do Kernel Linux, os scripts desenvolvidos em Shell Script e, por fim, a estratégia de versionamento utilizada.

2 Ambiente do Projeto: Docker

2.1 O que é o Docker

O Docker é uma plataforma de virtualização baseada em **containers**. Diferentemente de uma máquina virtual tradicional, que emula um hardware completo e exige um sistema operacional convidado completo (com seu próprio kernel), um container compartilha o kernel do sistema hospedeiro e isola apenas o espaço de usuário (bibliotecas, binários e arquivos de configuração). Isso resulta em imagens mais leves, criação quase instantânea e menor consumo de recursos quando comparado a uma máquina virtual completa.

A equipe optou pelo uso do Docker em vez de uma máquina virtual completa pelos seguintes motivos:

- **Portabilidade:** o ambiente pode ser reproduzido de forma idêntica em qualquer computador que tenha o Docker instalado, independentemente do sistema operacional do hospedeiro;
- **Reprodutibilidade:** o `Dockerfile` documenta de forma explícita e versionável todos os passos necessários para montar o ambiente;
- **Leveza:** a imagem ocupa menos espaço e inicializa em poucos segundos, facilitando testes repetidos durante o desenvolvimento dos scripts.

2.2 Configuração do Linux Mint no Docker

Para executar o Linux Mint em container, foi utilizada uma imagem baseada no Linux Mint 21.3 ("Virginia"), que por sua vez é construída sobre o Ubuntu 22.04 LTS. A partir dessa imagem base, foi criado um `Dockerfile` personalizado que:

1. Instala ferramentas auxiliares utilizadas pelos scripts da equipe (`htop`, `tree`, `neofetch`, `net-tools`, entre outras);
2. Copia os scripts Shell desenvolvidos pela equipe para dentro do container, no diretório `/projeto/scripts`;
3. Define como comando padrão (CMD) a execução do menu interativo `00_menu.sh`, que dá acesso a todas as demonstrações realizadas.

O trecho a seguir apresenta o `Dockerfile` utilizado:

Listing 1: `Dockerfile` do ambiente Linux Mint

```
1 FROM linuxmintd/mint21.3-amd64:latest
2
3 ENV DEBIAN_FRONTEND=noninteractive
4
5 RUN apt-get update && apt-get install -y \
6     htop tree neofetch net-tools procps \
7     nano less man-db kmod \
8     && apt-get clean && rm -rf /var/lib/apt/lists/*
9
10 WORKDIR /projeto
11
12 COPY scripts/ /projeto/scripts/
13
14 RUN chmod +x /projeto/scripts/*.sh
15
16 CMD ["/bin/bash", "/projeto/scripts/00_menu.sh"]
```

Para construir e executar o container, são utilizados os comandos:

Listing 2: Construção e execução do container

```
1 # Construir a imagem
2 docker build -t linuxmint-av2 -f docker/Dockerfile .
3
4 # Executar o container interativamente
5 docker run -it --rm linuxmint-av2
```

Alternativamente, a equipe disponibilizou um arquivo `docker-compose.yml`, permitindo a execução com um único comando:

Listing 3: Execução via Docker Compose

```
1 docker compose -f docker/docker-compose.yml run --rm linuxmint-av2
```

3 O Sistema Operacional Linux Mint

3.1 Visão geral

O **Linux Mint** é uma distribuição Linux de código aberto, lançada em 2006, derivada do Ubuntu (e, por consequência, do Debian). Seu principal objetivo é oferecer um sistema operacional elegante, moderno, confortável e de fácil utilização, sendo uma das distribuições mais populares para computadores desktop. Entre suas principais características, destacam-se:

- Utiliza o ambiente de área de trabalho **Cinnamon** (em sua edição principal), além de oferecer variações com os ambientes MATE e Xfce;
- Mantém compatibilidade total com o ecossistema de pacotes `.deb` e com o gerenciador de pacotes **APT**, herdado do Ubuntu/Debian;
- Inclui, por padrão, codecs multimídia e drivers proprietários, facilitando o uso imediato após a instalação;
- É amplamente utilizado como alternativa ao Windows, por usuários que buscam uma interface familiar com menor custo de aprendizado.

3.2 Estrutura do Sistema Operacional

De forma geral, qualquer sistema operacional do tipo Linux pode ser representado em camadas, conforme ilustrado na Figura 1:

1. **Hardware:** processador, memória, discos, placas de rede e demais dispositivos físicos;
2. **Kernel:** núcleo do sistema operacional, responsável por gerenciar diretamente o hardware e oferecer uma interface padronizada (chamadas de sistema) para os programas;
3. **Shell e bibliotecas do sistema:** interpretadores de comandos (como o Bash) e bibliotecas (como a `glibc`) que fazem a ponte entre as aplicações e o kernel;
4. **Aplicações:** programas utilizados pelo usuário final (navegadores, editores de texto, ferramentas administrativas etc.).

3.2.1 Hierarquia de diretórios (FHS)

A organização do sistema de arquivos do Linux Mint segue o padrão **FHS** (*Filesystem Hierarchy Standard*), comum à maioria das distribuições Linux. A Tabela 1 resume os diretórios mais importantes encontrados na raiz (`/`) do sistema.

Essa estrutura pode ser explorada de forma prática por meio do script `02_estrutura_arquivos.sh`, descrito na Seção 5.

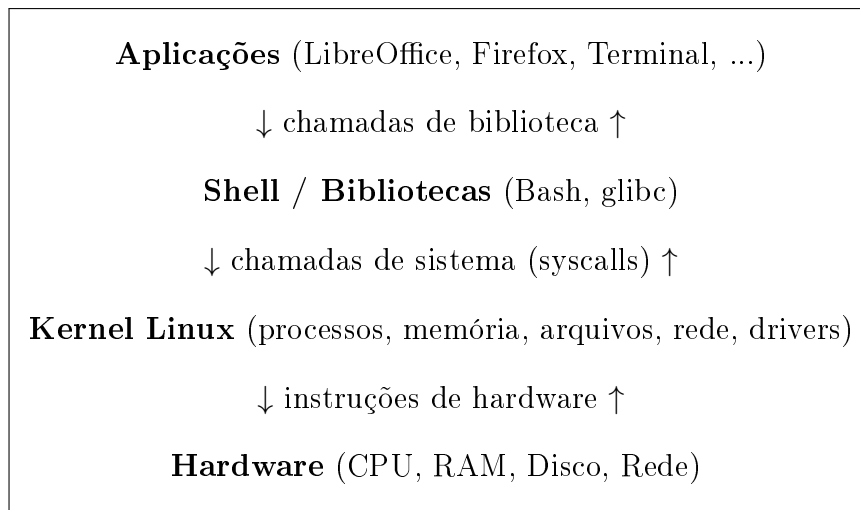


Figura 1: Camadas de um sistema operacional Linux

4 O Kernel Linux

4.1 Definição e tipo de kernel

O **kernel** é o componente central de um sistema operacional. Ele atua como intermediário entre o hardware e os programas em execução, sendo responsável por controlar o acesso aos recursos físicos da máquina (CPU, memória, dispositivos de entrada e saída) de forma segura e organizada.

Existem diferentes arquiteturas de kernel, sendo as principais:

- **Microkernel:** apenas funcionalidades mínimas (comunicação entre processos, gerenciamento básico de memória e escalonamento) ficam no espaço do kernel; drivers e serviços rodam em espaço de usuário, separados em processos isolados;
- **Kernel monolítico:** todos os serviços principais do sistema operacional (gerenciamento de processos, memória, sistema de arquivos, drivers, rede) executam no mesmo espaço de endereçamento privilegiado (kernel space).

O **Linux** adota uma abordagem **monolítica modular**: o núcleo é monolítico, mas grande parte de seus componentes (especialmente drivers de dispositivos) pode ser compilada como **módulos** (arquivos `.ko`), que são carregados e descarregados dinamicamente em tempo de execução, sem necessidade de reiniciar o sistema ou recompilar o kernel.

4.2 Principais funções do Kernel

1. **Gerenciamento de processos:** o kernel cria, escalona, suspende e finaliza processos. O componente responsável por decidir qual processo utilizará a CPU em cada instante é o *scheduler* (escalonador);
2. **Gerenciamento de memória:** controla a alocação de memória física e virtual para cada processo, implementa a memória *swap* e garante o isolamento entre os espaços de memória dos processos;

Diretório	Descrição
/bin	Binários essenciais (comandos básicos como <code>ls</code> , <code>cp</code> , <code>cat</code>).
/boot	Arquivos do kernel (<code>vmlinuz</code>), <code>initramfs</code> e do gerenciador de boot (GRUB).
/dev	Arquivos especiais que representam dispositivos de hardware.
/etc	Arquivos de configuração do sistema e dos aplicativos.
/home	Diretórios pessoais dos usuários.
/lib	Bibliotecas compartilhadas usadas pelos binários do sistema.
/proc	Sistema de arquivos virtual com informações do kernel e processos em execução.
/sys	Sistema de arquivos virtual com informações sobre dispositivos e drivers.
/tmp	Arquivos temporários.
/usr	Programas, bibliotecas e documentação para os usuários (maior parte do sistema).
/var	Dados variáveis: logs, caches, bancos de dados de pacotes.

Tabela 1: Principais diretórios do FHS no Linux Mint

3. **Gerenciamento de dispositivos (drivers):** comunica-se com o hardware por meio de drivers, que no Linux frequentemente são implementados como módulos do kernel;
4. **Sistema de arquivos:** implementa o suporte a diferentes formatos de sistemas de arquivos (`ext4`, `btrfs`, `NTFS`, etc.) e fornece chamadas de sistema (*syscalls*) como `open`, `read`, `write` e `close`;
5. **Comunicação entre processos e rede:** implementa mecanismos de IPC (*pipes*, sinais, memória compartilhada) e toda a pilha de protocolos de rede (TCP/IP).

4.3 Observando o Kernel na prática

Dentro do container Linux Mint, é possível observar informações fornecidas pelo kernel por meio de comandos simples e dos sistemas de arquivos virtuais `/proc` e `/sys`. Alguns exemplos utilizados no script `03_kernel_info.sh`:

Listing 4: Comandos para inspecionar o Kernel

```

1 # Versão do kernel em execução
2 uname -a
3
4 # Detalhes da versão do kernel (compilador, data)
5 cat /proc/version
6
7 # Módulos do kernel carregados atualmente
8 lsmod
9
10 # Parâmetros do kernel em tempo de execução

```

```

11 sysctl kernel.ostype kernel.osrelease kernel.version
12
13 # Dispositivos gerenciados pelo kernel
14 ls /sys/class

```

Esses comandos demonstram, na prática, que o kernel está constantemente expondo informações sobre o estado do hardware e dos processos através de arquivos virtuais, permitindo que ferramentas de espaço de usuário (como **ps**, **top** e **free**) leiam e apresentem esses dados de forma amigável.

5 Linguagem de Programação Utilizada: Shell Script

5.1 Justificativa

A linguagem escolhida pela equipe para a automação das demonstrações foi o **Shell Script**, na variante **Bash** (*Bourne Again Shell*), o interpretador de comandos padrão do Linux Mint. A escolha se justifica por:

- Ser a linguagem nativa de interação com o sistema operacional, permitindo acesso direto a comandos, arquivos de configuração e aos sistemas de arquivos virtuais `/proc` e `/sys`;
- Não exigir instalação de ferramentas adicionais, já que o Bash está presente em qualquer instalação padrão do Linux Mint (e, portanto, na imagem Docker utilizada);
- Ser amplamente utilizada para automação de tarefas administrativas, sendo um conteúdo diretamente relacionado à disciplina de Sistemas Operacionais.

5.2 Scripts desenvolvidos

Foram desenvolvidos seis scripts, organizados no diretório `scripts/` do repositório, descritos na Tabela 2.

Script	Descrição
00_menu.sh	Menu interativo, ponto de entrada do container, que permite escolher e executar as demais demonstrações.
01_info_sistema.sh	Exibe informações gerais do sistema: distribuição, <i>hostname</i> , tempo de atividade e arquitetura.
02_estrutura_arquivos.sh	Lista e descreve a estrutura de diretórios do sistema operacional (FHS).
03_kernel_info.sh	Exibe informações sobre o kernel: versão, módulos carregados, parâmetros de execução e explica suas funções.
04_processos_memoria.sh	Demonstra o gerenciamento de processos e memória realizado pelo kernel, utilizando ps , free e pstree .
05_pacotes.sh	Demonstra o funcionamento do gerenciador de pacotes APT/dpkg, utilizado pelo Linux Mint.

Tabela 2: Scripts desenvolvidos pela equipe

A título de exemplo, o trecho a seguir apresenta o script `02_estrutura_arquivos.sh`, responsável por listar e descrever os diretórios da raiz do sistema:

Listing 5: Trecho do script `02_estrutura_arquivos.sh`

```
1 #!/bin/bash
2 echo "Conteúdo da raiz '/' : "
3 ls -lh /
4
5 echo "Descrição de cada diretório principal: "
6 cat <<'TEXT0'
7 /bin      -> Binários essenciais usados por todos os usuários
8 /boot     -> Arquivos necessários para o boot (kernel, GRUB)
9 /etc      -> Arquivos de configuração do sistema
10 /home     -> Diretórios pessoais dos usuários
11 /proc     -> Informações do kernel e processos (sistema virtual)
12 /usr      -> Programas e bibliotecas para os usuários
13 /var      -> Dados variáveis: logs, caches, etc.
14 TEXT0
```

Todos os scripts foram testados dentro do container Linux Mint e podem ser executados individualmente ou através do menu principal (`00_menu.sh`), conforme demonstrado durante a apresentação.

6 Versionamento com Git e GitHub

Todo o material produzido pela equipe (Dockerfile, scripts, relatório em $\text{\LaTeX}$ e demais arquivos de apoio) foi versionado utilizando o sistema de controle de versão **Git** e hospedado em um repositório do **GitHub**.

A organização do repositório segue a estrutura apresentada a seguir:

Listing 6: Estrutura do repositório no GitHub

```
1 projeto-av2-linux-mint/
2 |-- README.md
3 |-- .gitignore
4 |-- docker/
5 |   |-- Dockerfile
6 |   |-- docker-compose.yml
7 |   '-- .dockerignore
8 |-- scripts/
9 |   |-- 00_menu.sh
10 |   |-- 01_info_sistema.sh
11 |   |-- 02_estrutura_arquivos.sh
12 |   |-- 03_kernel_info.sh
13 |   |-- 04_processos_memoria.sh
14 |   '-- 05_pacotes.sh
15 '-- relatorio/
16 |   |-- relatorio.tex
17 |   '-- relatorio.pdf
```

O fluxo de trabalho adotado pela equipe seguiu, de forma simplificada, os seguintes passos:

1. Criação do repositório no GitHub e clonagem local (`git clone`);

2. Desenvolvimento incremental do `Dockerfile` e dos scripts, com commits descritivos para cada funcionalidade adicionada;
3. Testes do ambiente Docker a cada alteração relevante;
4. Escrita e atualização do relatório em $\text{\LaTeX}$ em paralelo ao desenvolvimento técnico;
5. Envio (*push*) das alterações para o repositório remoto, mantendo o histórico de versões disponível para consulta.

O link do repositório utilizado pela equipe é: <https://github.com/USUARIO/projeto-av2-linux-mint> (*substituir pelo endereço real do repositório da equipe*).

7 Conclusão

O desenvolvimento deste projeto permitiu à equipe consolidar, de forma prática, diversos conceitos fundamentais da disciplina de Sistemas Operacionais. A utilização do Docker possibilitou a criação de um ambiente isolado, portátil e facilmente reproduzível, contendo o Linux Mint, sem a necessidade de uma instalação tradicional em máquina física ou virtual completa.

Através da exploração da estrutura de diretórios (FHS) e dos mecanismos oferecidos pelo Kernel Linux (gerenciamento de processos, memória, dispositivos e arquivos), foi possível compreender, na prática, como as camadas de hardware, kernel, shell e aplicações se relacionam para formar um sistema operacional funcional.

Por fim, o uso de Shell Script para automatizar as demonstrações reforçou a importância dessa linguagem como ferramenta de administração de sistemas Linux, enquanto o versionamento com Git e GitHub garantiu organização, rastreabilidade e colaboração eficiente entre os membros da equipe durante todo o desenvolvimento do projeto.

8 Referências

1. LINUX MINT TEAM. *Linux Mint Documentation*. Disponível em: <https://linuxmint.com>.
2. THE LINUX KERNEL ORGANIZATION. *The Linux Kernel documentation*. Disponível em: <https://kernel.org>.
3. DOCKER INC. *Docker Documentation*. Disponível em: <https://docs.docker.com>.
4. FILESYSTEM HIERARCHY STANDARD GROUP. *Filesystem Hierarchy Standard (FHS)*. Disponível em: <https://refspecs.linuxfoundation.org/fhs.shtml>.
5. TANENBAUM, A. S.; BOS, H. *Modern Operating Systems*. 4. ed. Pearson, 2014.